



操作系统课程实验

——从0到1的小型内核实现

2025年 秋季学期

lab-6 进程管理模块

1. 进程的切换
2. fork+wait+sleep+exit 系统调用
3. 时钟中断管理进程调度
4. 总结

1. 进程的切换

本质：

操作系统将**CPU的控制权**从一个正在运行的进程转移到另一个进程的过程。

如何理解进程拥有cpu的控制权？

直接说结论：

该进程的**内核态上下文**（包括寄存器、页表、内核栈等）已被加载到 CPU，CPU当前执行的指令流属于该进程。

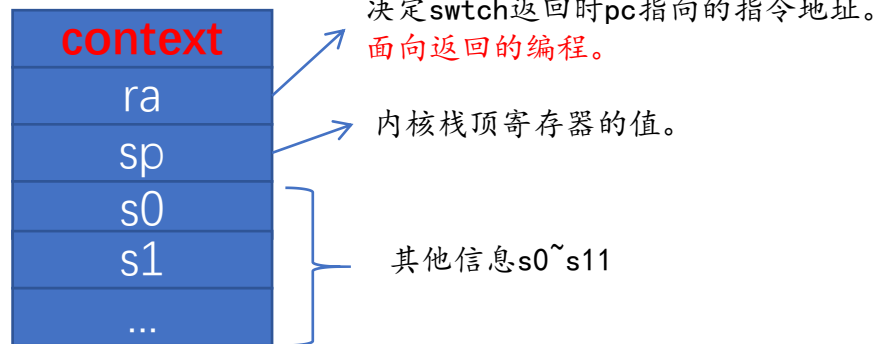
这就引出了另一个结论：

进程的切换**只能在内核态进行**。

否则软件开发人员就可以一直占用cpu资源。

内核态上下文？

理解context：



cpu在内核态的时候有两类执行对象：

➤ 内核态的进程。

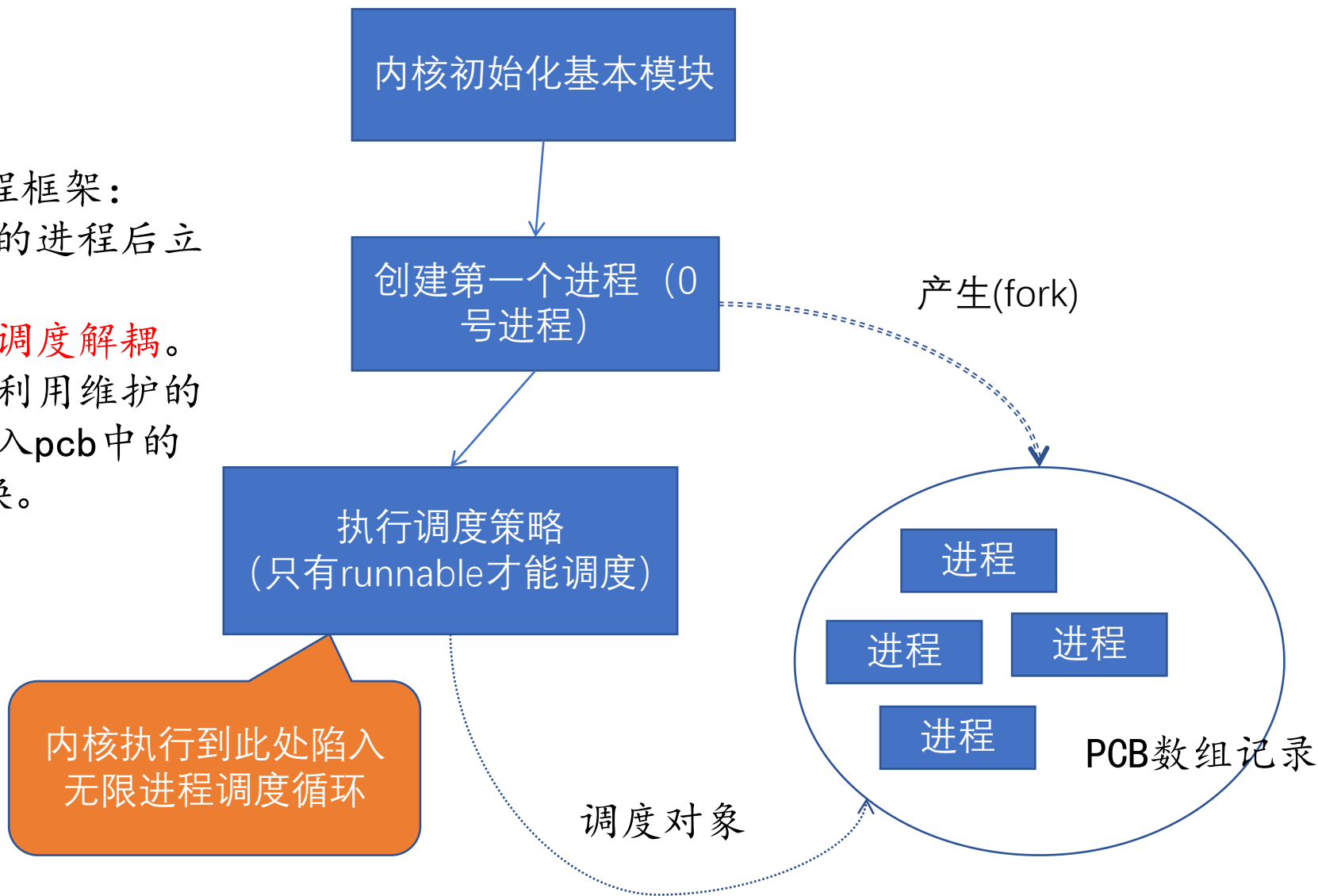
➤ 内核本身（cpu上下文）。

这两类执行对象都有自己的上下文。

1. 进程的切换

本次实验内核管理进程的流程框架：

- 先前的实验中我们创建新的进程后立即将其调度执行。
- 如今我们将进程的**创建与调度解耦**。
- 内核中的进程调度模块是利用维护的**pcb数组**。通过给swtch传入pcb中的ctx字段来进行进程的切换。



2. fork + wait+sleep+exit系统调用

什么是fork?

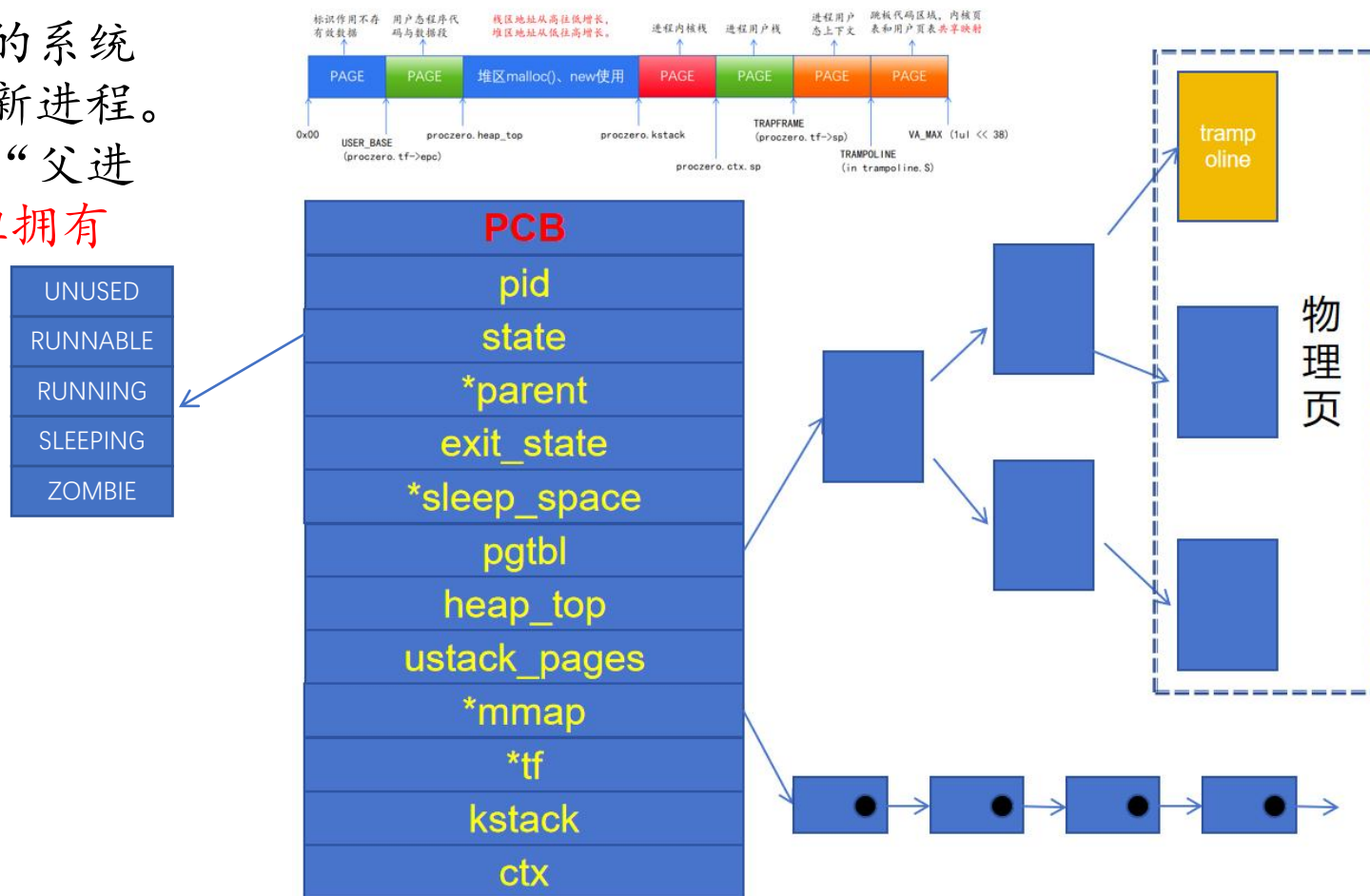
fork 是 Unix/Linux 系统中一个核心的系统调用 (system call)，用于创建一个新进程。新进程是调用 fork 的原进程（称为“父进程”）的副本，二者几乎完全相同，但拥有独立的执行环境。

如何实现fork?

- 地址空间拷贝。
- 堆区链表的拷贝。
- 上下文的拷贝。
- 状态的设置，父进程字段设置。
- 子进程fork函数返回值的设置。

fork之后，pcb数组中就多了一个可以调度运行的进程。

fork之后，进程就有了“进程树”的结构，对资源管理构成挑战。



2. fork + wait+sleep+exit 系统调用

wait系统调用:

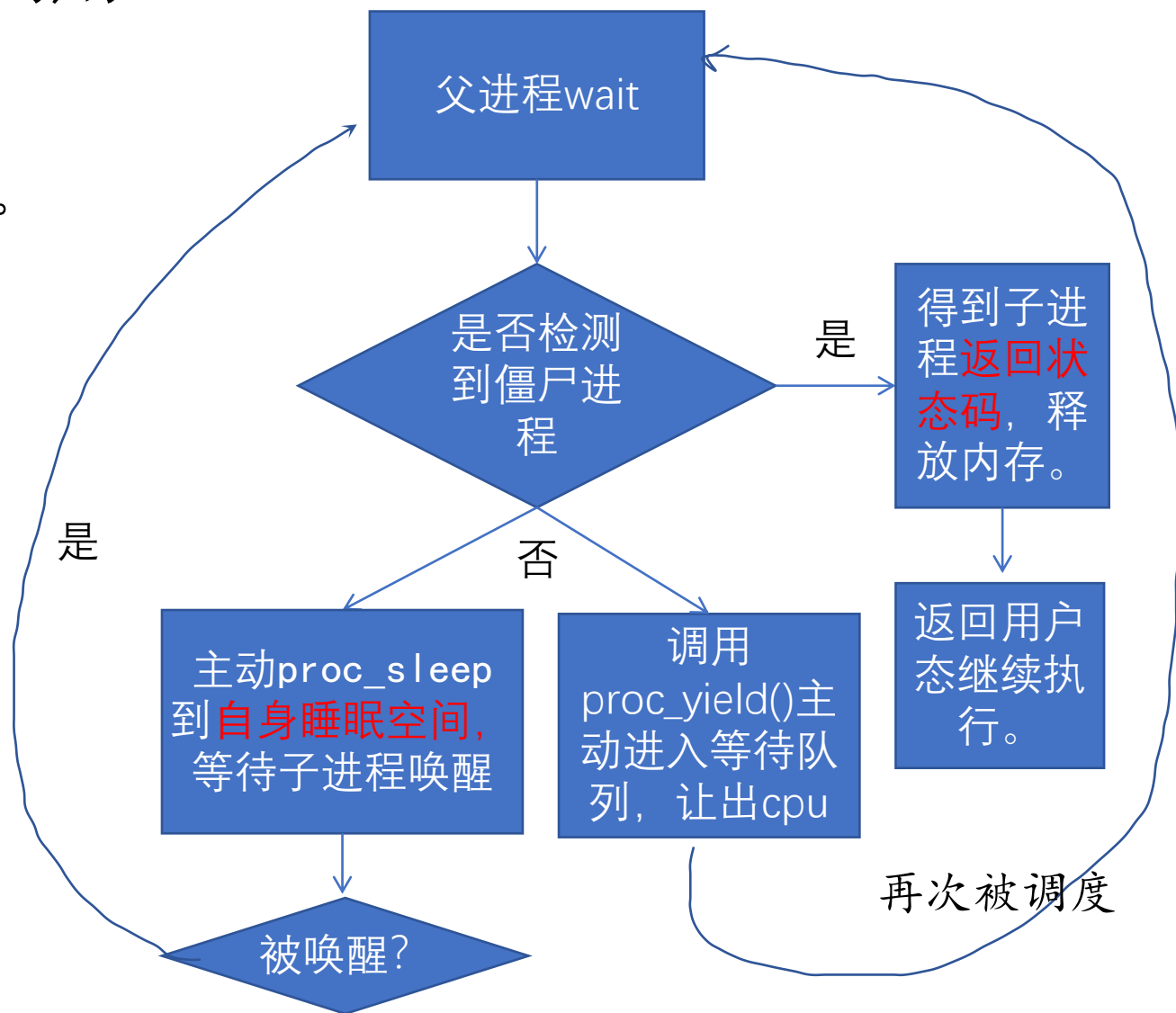
- 等待子进程exit进入**僵尸态**，回收其资源。
- 没有子进程退出就主动睡眠，**睡眠空间**指向自己。

sleep系统调用:

- 底层仍然调用proc_sleep，但是睡眠空间变成时钟中断的tick变量地址。
- 时钟中断通过tick值和睡眠时间对比来决定唤醒时机。

exit系统调用:

- 设置自身为僵尸态，不再参与调度。
- 唤醒父进程进行资源回收，而不是自己释放自己。



3. 时钟中断管理进程调度

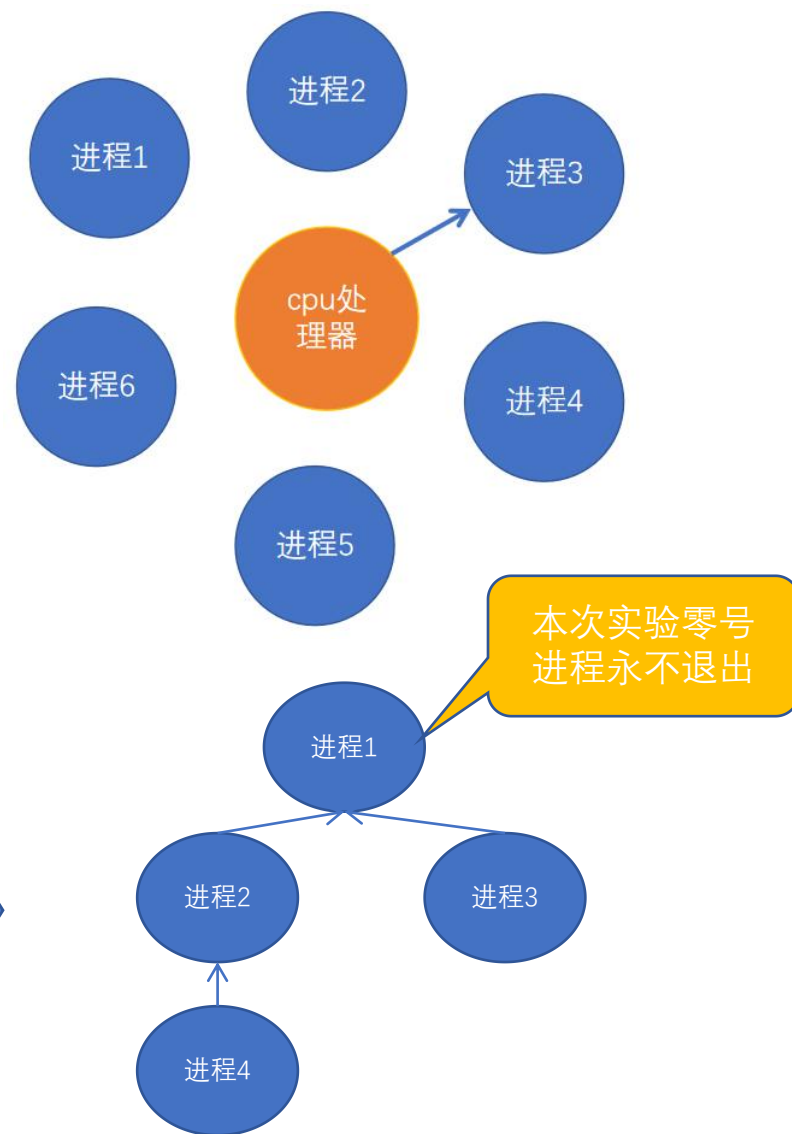
时钟中断管理进程调度的必要性：

- 由于大部分被情况进程不会主动让出cpu，如果一个恶意进程一直占用CPU而不调用 `proc_yield()` 或 `proc_sleep()`，那么CPU事实上就被它独占了而其他进程只能一直等待。
- 为了避免这样的事情发生，我们应该提供一种强制进程交出CPU使用权的机制：可以利用时钟中断实现这件事：在发生时钟中断后调用 `proc_yield()`。

如何创造多个进程测试？

- 用户进程多次fork即可。
- 想一想连续n个fork会产生几个子进程，能否画出进程树？

```
void test2()
{
    syscall(SYS_fork);
    syscall(SYS_fork);
}
```



4. 总结

- 我们介绍了进程切换的本质：cpu在内核态加载运行不同执行对象的内核态上下文context，因为在内核态所有执行对象公用同一张内核页表，所以切换开销相对较小，请认真阅读swtch。
- 我们介绍了fork系统调用，介绍了一个进程如何深度拷贝自己pcb所维护的进程元信息来创建子进程。
- 我们介绍了wait, sleep, exit系统调用，了解了进程如和通过这些系统调用实现自身状态的转变，以及知道他们之间的制约和配合的关系来管理和回收进程资源。
- 最后我们介绍了基于时钟中断的时间片轮转调度算法，让每个进程公平的使用cpu资源，防止进程不主动让出的情况。（目前项目中的进程只能在用户态相应时钟中断，进入内核态后时钟中断无法正常执行，后续会修复。）